



Log Analysis and Failure Prediction for Data Pipeline Batch Jobs Using XGBoost Algorithm

Ajibola, Mojolaoluwa David
| 22CG031820

Supervisor: Dr Iheanetu
Olamma | Department of
Computer and Information
Sciences

Presentation Outline

- 1. Background and problem context
- 2. Aim, five objectives, and literature gap
- 3. Methodology, data sources, and model validation
- 4. System architecture, risk logic, and implementation
- 5. Feature-importance findings and evaluation results
- 6. Dashboard evidence, discussion, limitations, recommendations, and conclusion

Background

- Modern ETL pipelines move large volumes of data across cloud storage, logs, scheduled transformations, and downstream analytics.
- Batch jobs often run unattended, so failures can remain hidden until reports, analytics, or dependent jobs are affected.
- Machine learning can support early warning by converting historical and live execution evidence into interpretable risk signals.
- This project narrows that idea to log analysis and batch-job failure prediction using XGBoost and supporting model comparisons.

Statement of the Problem

- Current monitoring often shows logs or completed failure alerts after a batch job has already degraded.
- Manual log analysis keeps engineers in reactive maintenance rather than proactive intervention.
- Pipeline defects can cascade into data-quality issues, missing outputs, analytics delays, and business decision errors.
- The gap: an integrated workflow that learns failure patterns, classifies live risk, and presents evidence for timely action.

Aim and Objectives

1. Aim: Design and develop a log analysis and failure prediction system for ETL batch jobs using XGBoost.
2. 1. Identify and characterize common failure patterns in data pipeline batch jobs.
3. 2. Compare multiple machine learning algorithms for failure prediction.
4. 3. Develop a prediction model that continuously monitors batch pipeline execution.
5. 4. Evaluate the system using Precision, Recall, F1-score and supporting metrics.
6. 5. Visually represent the monitoring workflow using a dashboard and orchestration evidence.

Literature Gap

- Existing studies show that machine learning can support job-failure and log-anomaly detection.
- DeepLog demonstrates sequence-based log anomaly detection, but not an ETL-facing monitoring workflow.
- Cloud job-failure prediction studies show model value, but performance depends on workload source and features.
- Recent log-anomaly research emphasizes usability, trust, and source adaptation.
- Project response: source-specific models plus live ETL attachment, dashboard review controls, and generated reports.

Methodological Alignment

- Research design: applied experimental machine learning plus prototype-based software development.
- Objective 1: extract severity, keyword, retry, timing, resource, and task-failure signals.
- Objective 2: compare baseline, Logistic Regression, Decision Tree, Random Forest, HistGradientBoosting, and XGBoost.
- Objective 3: integrate the selected model with a live runner and predictor plugin.
- Objective 4-5: evaluate metrics and visualize workflow through Streamlit, reports, and Airflow evidence.

Data Sources and Sampling

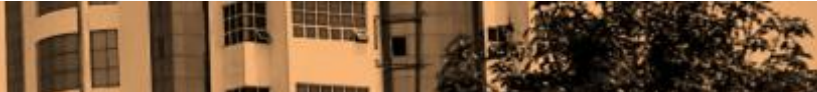
- HDFS / ETL-compatible logs: primary live dashboard schema and controlled ETL demonstration.
- Alibaba PAI GPU trace: bounded production-style benchmark with 5,000 jobs and 273 failures.
- Google Borg 2019 trace: bounded benchmark with 10,000 records and 2,299 failures.
- Synthetic ETL evidence: 100 ETL records and 400 early-warning snapshots for repeatable defence demonstration.
- Large raw traces are kept out of deployment; saved models and comparison artifacts are retained.

Feature Engineering

- Raw logs are converted into structured features instead of being passed directly into XGBoost.
- Severity profile: INFO, WARN, ERROR, FATAL counts and rates.
- Failure indicators: exception, failed, timeout, retry, stopped, invalid, and error-burst signals.
- Workload context: task counts, failed-task counts, scheduling delay, requested CPU/memory/GPU, and runtime samples.
- Feature sets remain source-specific to avoid false merged-model claims.

Model Development and Validation

- XGBoost configuration: 300 estimators, max_depth 5, learning_rate 0.08, subsample 0.8, colsample_bytree 0.8, random_state 42.
- Split strategy: 25% held-out testing with group-held-out splitting where process/job groups exist.
- Cross-validation: 5-fold StratifiedGroupKFold when grouped snapshots are available; otherwise stratified row folds.
- Evaluation: accuracy, precision, recall, F1, ROC-AUC, PR-AUC, MCC, specificity, log loss, inference latency, and confusion matrix.
- Artifacts: saved model files, metrics, charts, and comparison outputs.



System Architecture

- External ETL process emits extract, validate, transform, load, and report logs.
- Live attachment runner captures stdout, stderr, and resource samples.
- Log adapter normalizes records for streaming feature extraction.
- Predictor plugin applies the trained artifact, stores alerts, and prepares report payloads.
- Streamlit dashboard presents stages, logs, risks, charts, review controls, and downloads.



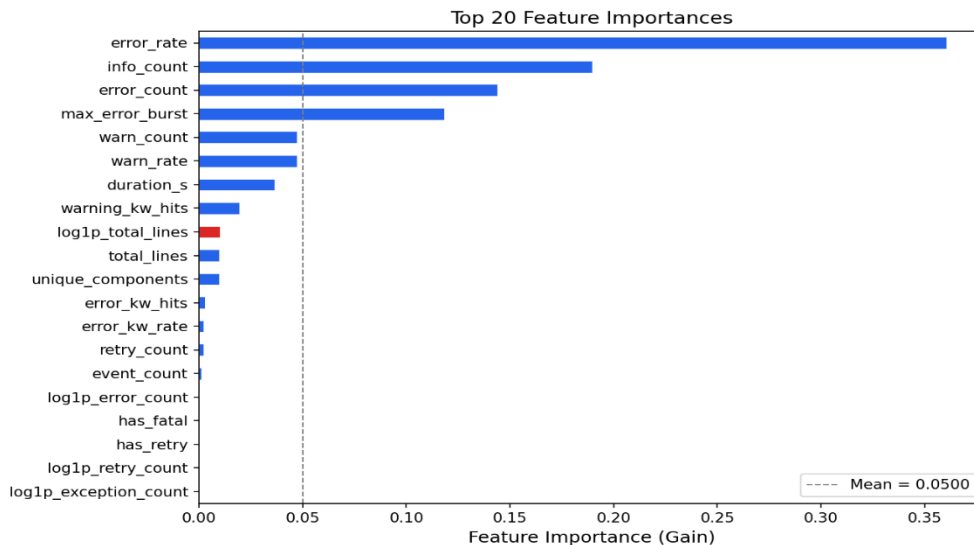
Risk Classification and Intervention

- LOW risk: probability below 0.40; continue monitoring.
- MEDIUM risk: probability from 0.40 to below 0.70; review warnings and context.
- HIGH risk: probability at or above 0.70; trigger intervention review.
- Review mode keeps human judgement in the loop before stopping an execution.
- Auto-stop remains configurable for environments where the cost of continuation is too high.



Feature-Importance Findings

- Objective 1 is closed by the saved XGBoost feature-importance evidence.
- Top warning patterns: error_rate, info_count, error_count, max_error_burst, warn_count, and warn_rate.
- Interpretation: failure risk is driven by error concentration, abnormal bursts, warning density, and execution-volume signals.
- These features are observable while a batch job is still running.

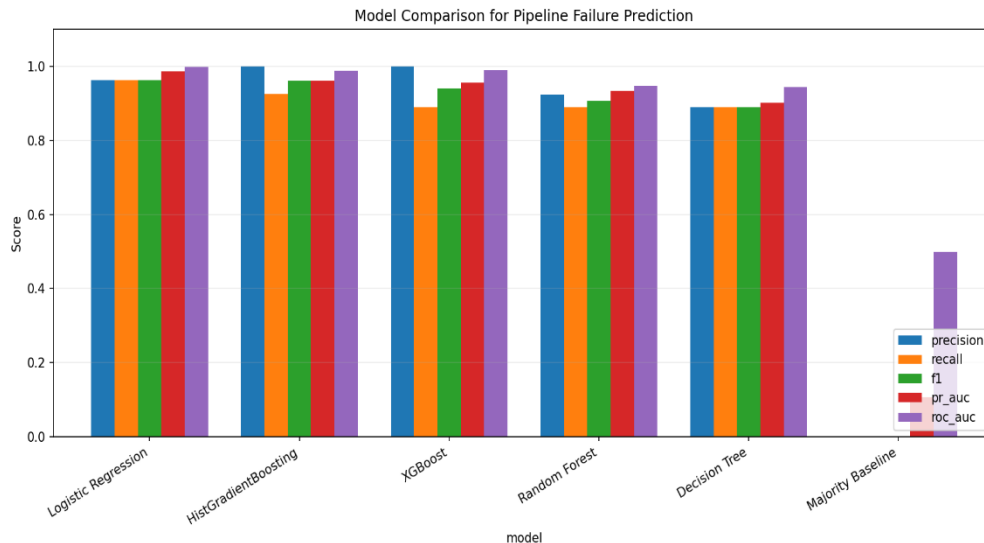


Model Evaluation Results

- ETL/HDFS-compatible held-out model:
- Accuracy = 0.9250 | Precision = 0.8421 | Recall = 1.0000 | F1-score = 0.9143
- ROC-AUC = 0.9883 | PR-AUC = 0.9704 | MCC = 0.8584 | Log loss = 0.1222
- Confusion matrix: TP 16, TN 21, FP 3, FN 0.
- Interpretation: strong controlled-demo evidence, but not a universal production guarantee.

Model Comparison

- The deployed model was compared against simpler and ensemble alternatives.
- Comparison set: Majority Baseline, Logistic Regression, Decision Tree, Random Forest, HistGradientBoosting, and XGBoost.
- XGBoost remains the implemented monitor model because it fits the engineered-feature artifact path.
- Benchmark evidence shows model choice should remain source-specific.



Dashboard and Output Evidence

- Dashboard evidence: run controls, stage board, log stream, risk KPIs, resource samples, and model comparison views.
- HIGH-risk review controls allow the operator to continue or stop a risky run.
- Report generator produces JSON and HTML summaries with alerts, logs, risk counts, high-risk items, and resource evidence.
- Airflow DAG shows orchestration evidence through prepare, run, evaluate, and publish tasks.
- Render deployment hosts the Streamlit dashboard as a lightweight web demonstration.



Discussion of Findings

- The ETL/HDFS-compatible result supports the live demonstration because the scenarios are controlled and observable.
- Alibaba and Google benchmark results are more modest, confirming that workload traces are harder and source-dependent.
- Accuracy alone is insufficient under class imbalance; recall, precision, F1-score, PR-AUC and MCC are more useful for failure detection.
- Compared with DeepLog and cloud job-failure studies, this project adds an ETL-facing dashboard, intervention logic, and audit reports.

Limitations

- The live ETL process is controlled and does not reproduce all enterprise dependencies.
- Alibaba and Google experiments use bounded samples rather than full-trace production validation.
- Risk thresholds are explainable for defence but require production cost calibration.
- Render deployment covers the Streamlit dashboard, not Airflow workers, authentication, persistent storage, or continuous retraining.
- Formal explainability such as SHAP is future work.

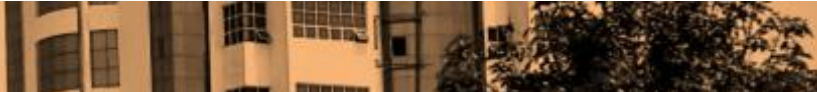
Recommendations and Future Work

- Package the predictor as a formal plugin or service with a documented API and configuration.
- Add persistent storage for monitoring sessions, reports, thresholds, and model-version tracking.
- Validate on larger and real organizational pipeline logs after privacy review.
- Add authentication and access control before any non-defence deployment.
- Compare sequence-aware log models against the current engineered-feature approach.



Final Conclusion

- The project delivers a defensible final-year prototype for log analysis and failure prediction in ETL batch jobs.
- Its main contribution is the integration of machine learning evidence with an operational workflow: logs are observed, features are extracted, risk is classified, HIGH risk triggers review, and reports are generated.
- The system is not presented as a finished production platform, but it is a strong foundation for proactive data-pipeline reliability management.



References

- Akash, V. C., & Charate, P. C. (2025). Proactive data pipeline maintenance via ML-driven anomaly detection.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system.
- Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). DeepLog.
- Foidl, H., Golendukhina, V., Ramler, R., & Felderer, M. (2024). Data pipeline quality.
- Jassas, M. S., & Mahmoud, Q. H. (2022). Job failure prediction in cloud computing.
- Ma, X. et al. (2024). Practitioners' expectations on log anomaly detection.
- Wang, Y. et al. (2024). Cross-system software log-based anomaly detection.
- Zhang, L. et al. (2024). Multivariate log-based anomaly detection for distributed database.



Thank You for Listening

Questions, comments,
and suggestions

